Praktikum JavaScript & TypeScript

Modul 6 - Modules dan Standard Library

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, peserta mampu:

- Memecah program menjadi beberapa file dengan `export` dan `import`
- Membulatkan dan menghitung dengan `Math`, serta memformat angka dengan `Intl.NumberFormat`
- Membuang duplikat dengan `Set` dan mencari data dengan cepat lewat `Map`
- Membuat dan memformat tanggal dengan `Date` tanpa terjebak bulan yang dimulai dari 0
- Melempar error dengan `throw` dan menangkapnya dengan `try...catch`
- Memilih di mana error dilempar dan di mana ditangkap

2. Dasar Teori

2.1. Modules: Memecah Program Menjadi Beberapa File

Untuk apa: program yang besar sulit dibaca jika semuanya ada di satu file. Module memisahkan kode per tanggung jawab. Contohnya satu file untuk perhitungan kasir, satu file untuk tampilan. Mirip memisahkan stored procedure ke dalam skema sendiri di database: yang lain cukup memanggil, tidak perlu tahu isinya.

Cara kerja: setiap file `.js` adalah satu module. Semua yang ada di dalamnya **privat** secara bawaan. Hanya yang diberi `export` yang bisa dipakai file lain, dan file lain memakainya dengan `import`.

```
export const PI_KASAR = 3.14;

export function luasLingkaran(r) {
  return PI_KASAR * r * r;
}

function bantuRahasia() {        // tidak di-export: hanya bisa dipakai di file ini
  return "rahasia";
}
```

Listing 1: geometri.js: module yang meng-export

```
import { luasLingkaran, PI_KASAR } from "./geometri.js";

console.log(luasLingkaran(10));   // 314
console.log(PI_KASAR);            // 3.14
```

Listing 2: utama.js: file yang meng-import

Aturan penulisan `import` di Node:

- Nama di dalam `{ }` harus **persis sama** dengan nama yang di-export. Ini disebut *named export*.
- Path diawali `./` (folder yang sama) atau `../` (folder induk), dan **wajib** memakai ekstensi `.js`.
- Repo ini memakai `"type": "module"` di `package.json`, jadi semua file `.js` dianggap module.

2.1.1. Import dijalankan sebelum baris lain

Node memproses module dalam dua tahap:

1. **Menghubungkan.** Node membaca semua baris `import` di seluruh file, memuat file yang di-import, lalu memeriksa apakah semua nama yang diminta benar-benar di-export.
2. **Menjalankan.** Baru setelah semua `import` cocok, kode dijalankan dari baris pertama.

Akibatnya, jika satu nama yang di-import tidak ada, **tidak ada satu baris pun** yang dijalankan. Ini berlaku juga untuk `console.log` yang ditulis di atas `import`. Karena itu `import` biasanya ditulis di baris paling atas.

Kesalahan	Pesan error
Nama tidak di-export, atau salah tulis	`SyntaxError: The requested module './geometri.js' does not provide an export named 'luas'`
Ekstensi `.js` lupa ditulis	`Error [ERR_MODULE_NOT_FOUND]: Cannot find module '.../geometri' imported from ...`
Memakai `require` (gaya lama CommonJS)	`ReferenceError: require is not defined in ES module scope, you can use import instead`

> **✎ Catatan**
>
> Ada juga *default export*: `export default function ...`, yang di-import tanpa kurung kurawal (`import luas from "./geometri.js"`). Di modul ini kita hanya memakai named export, karena namanya selalu tertulis jelas di kedua file.

2.2. Math: Pembulatan dan Perhitungan

`Math` adalah object bawaan yang berisi fungsi matematika. Kamu tidak membuatnya dengan `new`, cukup memanggil `Math.nama(...)`.

2.2.1. Empat cara membulatkan

Fungsi	`2.5`	`-2.5`	`2.1`	Aturan
`Math.round`	`3`	`-2`	`2`	ke bilangan bulat terdekat; tepat `.5` naik ke arah positif
`Math.floor`	`2`	`-3`	`2`	selalu ke bawah (ke arah minus)
`Math.ceil`	`3`	`-2`	`3`	selalu ke atas (ke arah plus)
`Math.trunc`	`2`	`-2`	`2`	buang bagian desimal

Pilih dari kalimat aturannya: “dibulatkan ke bawah” berarti `floor`, “minimal” atau “tidak kurang dari” berarti `ceil`.

2.2.2. Pola: membulatkan ke kelipatan

`Math.ceil` dan `Math.floor` hanya membulatkan ke bilangan bulat. Untuk membulatkan ke kelipatan tertentu, bagi dulu, bulatkan, lalu kalikan kembali.

```
// Parkir dihitung per jam penuh: 61 menit dibayar 2 jam.
function jamParkir(menit) {
  return Math.ceil(menit / 60);
}
console.log(jamParkir(61), jamParkir(60), jamParkir(5));   // 2 1 1

// Dibulatkan ke kelipatan 15 menit ke atas
const blok = Math.ceil(50 / 15) * 15;
console.log(blok);   // 60
```

Listing 3: tarif parkir per jam

Jalannya `Math.ceil(50 / 15) * 15`:

Langkah	Nilai
`50 / 15`	`3.333...` (berapa blok 15 menit)
`Math.ceil(...)`	`4` (blok terakhir yang terpakai sebagian dihitung penuh)
`* 15`	`60` (kembali ke satuan menit)

2.2.3. Fungsi lain yang sering dipakai

```
const nilai = [3, 9, 4];
console.log(Math.max(...nilai));   // 9   (spread: Math.max(3, 9, 4))
console.log(Math.max(nilai));      // NaN (array tidak dibuka, dianggap satu nilai)
console.log(Math.min(...nilai));   // 3
console.log(Math.abs(-7));         // 7
```

Listing 4: max, min, abs

> **⚠ Perhatian**
>
> Angka desimal di komputer tidak selalu tepat: `0.1 + 0.2` menghasilkan `0.30000000000000004`. Untuk uang, simpan dalam satuan terkecil berupa bilangan bulat (rupiah, bukan ribuan rupiah dengan desimal). Jika terpaksa memakai desimal, bulatkan saat menampilkan: `Math.round((0.1 + 0.2) * 100) / 100` menghasilkan `0.3`.

2.3. Intl.NumberFormat: Format Angka Sesuai Negara

Untuk apa: menampilkan angka dengan pemisah ribuan dan desimal yang lazim di suatu negara. Indonesia memakai titik untuk ribuan dan koma untuk desimal.

Cara kerja: buat formatter dengan `new Intl.NumberFormat(locale, opsi)`, lalu panggil `.format(angka)`. Hasilnya string.

```
const fmt = new Intl.NumberFormat("id-ID");
console.log(fmt.format(1234567.891));   // "1.234.567,891"
console.log(fmt.format(500));           // "500"

const duaDesimal = new Intl.NumberFormat("id-ID", { maximumFractionDigits: 2 });
console.log(duaDesimal.format(1234.5)); // "1.234,5"

const berat = `${fmt.format(12500)} kg`;
console.log(berat);                     // "12.500 kg"
```

Listing 5: format angka Indonesia

> **⚠ Perhatian**
>
> Ada opsi `{ style: "currency", currency: "IDR" }` yang menghasilkan `"Rp 15.000"`. Spasi di antara `Rp` dan angka adalah *non-breaking space* (karakter ` `), bukan spasi biasa. Hasilnya terlihat sama, tetapi `===` dengan `"Rp 15.000"` yang diketik biasa bernilai `false`. Jika format teks harus persis, susun sendiri dari `format(...)` dan template literal.

2.4. Set: Kumpulan Nilai Tanpa Duplikat

Untuk apa: menyimpan nilai unik, atau membuang duplikat dari array. Di SQL: `SELECT DISTINCT`.

Cara kerja: `new Set(array)` membuat Set dari isi array. Nilai yang sama hanya disimpan sekali, dengan urutan kemunculan pertama. Set bukan array, jadi tidak punya `map` atau index. Ubah kembali menjadi array dengan spread `[...set]`.

```
const s = new Set(["a", "b", "a"]);
console.log(s.size);          // 2
console.log(s.has("a"));      // true   (seperti includes)
s.add("c");                   // tambah
s.add("a");                   // sudah ada: diabaikan
s.delete("b");                // hapus
console.log([...s]);          // ["a", "c"]

const unik = [...new Set([3, 1, 3, 2, 1])];
console.log(unik);            // [3, 1, 2]  (urutan kemunculan pertama)
```

Listing 6: Set

Di Modul 3, daftar kategori unik dikumpulkan dengan `for...of`, `includes`, dan `push`. Dengan Set, cukup satu baris: ambil kolomnya dengan `map`, masukkan ke Set, lalu ubah kembali menjadi array.

```
const pelanggan = [
  { nama: "Ani", kota: "Solo" },
  { nama: "Budi", kota: "Semarang" },
  { nama: "Cici", kota: "Solo" },
];
const kota = [...new Set(pelanggan.map((p) => p.kota))];
console.log(kota);   // ["Solo", "Semarang"]
```

Listing 7: daftar kota unik

Bentuk data di setiap langkah: `pelanggan` (array object), lalu `map` (array string dengan duplikat), lalu `new Set(...)` (Set tanpa duplikat), lalu `[...]` (array string).

2.5. Map: Kamus dengan Pencarian Cepat

Untuk apa: menyimpan pasangan key dan value, lalu mencari value lewat key dengan cepat. Contohnya mencari produk lewat id, atau mahasiswa lewat NIM.

Cara kerja: `new Map()` membuat Map kosong. `set(key, value)` menyimpan, `get(key)` mengambil (atau `undefined` jika tidak ada), `has(key)` memeriksa, dan `size` memberi jumlahnya. Key yang sama ditimpa.

```
const m = new Map();
m.set("a", 1).set("b", 2);    // set bisa di-chain
m.set("a", 9);                // key sama: ditimpa
console.log(m.size);          // 2
console.log(m.get("a"));      // 9
console.log(m.get("z"));      // undefined
console.log(m.has("b"));      // true
for (const [key, value] of m) {
  console.log(key, value);    // a 9, lalu b 2
}
```

Listing 8: Map

2.5.1. Map atau object?

Hal	Object `{}`	`Map`
Tipe key	selalu diubah menjadi string: `o[1]` sama dengan `o["1"]`	apa saja, tanpa diubah: `get(1)` berbeda dengan `get("1")`
Jumlah isi	`Object.keys(o).length`	`m.size`
Diulang	`Object.entries(o)` atau `for...in`	langsung `for...of`
Cocok untuk	satu data dengan field tetap, dan hasil JSON	kamus atau index yang isinya ditambah dan dicari terus

2.5.2. Pola: index lewat id

Di Modul 3, produk dicari dengan `find`. `find` memeriksa elemen satu per satu dari depan sampai ketemu. Untuk data besar yang sering dicari, buat **index** sekali di awal, lalu cari lewat `get`.

```
const mahasiswa = [
  { nim: "A11", nama: "Ani" },
  { nim: "A12", nama: "Budi" },
  { nim: "A13", nama: "Cici" },
];

const indexNim = new Map();
for (const m of mahasiswa) {
  indexNim.set(m.nim, m);          // key: nim, value: object mahasiswa
}

console.log(indexNim.get("A12").nama);   // "Budi"
console.log(indexNim.get("Z99"));        // undefined
```

Listing 9: index mahasiswa lewat NIM

Berapa langkah pencarian yang dilakukan?

Cara	Satu pencarian di 10.000 data	50 pencarian
`array.find`	sampai 10.000 perbandingan	sampai 500.000
`map.get`	kira-kira 1 langkah	kira-kira 50

Map menyimpan isinya dengan *hash table*: key diubah menjadi alamat penyimpanan, jadi `get` langsung melompat ke tempatnya tanpa memeriksa elemen lain. Di database, ini sama dengan `CREATE INDEX` atau index bawaan primary key. Tanpa index, database harus memindai seluruh tabel (*full table scan*), sama seperti `find`. Biaya membuat index hanya dibayar sekali (satu perulangan), lalu setiap pencarian menjadi cepat.

2.6. Date: Tanggal dan Waktu

Untuk apa: menyimpan tanggal, misalnya tanggal transaksi atau jatuh tempo.

Cara kerja: `new Date(tahun, bulan, hari)` membuat tanggal. `new Date()` tanpa argumen berisi waktu sekarang. Komponennya dibaca dengan getter.

> ⚠ **Perhatian**
>
> **Bulan dimulai dari 0.** Januari adalah `0`, Desember adalah `11`. Ini berlaku saat membuat (`new Date(2026, 0, 31)` adalah 31 Januari) dan saat membaca (`getMonth()` untuk Januari mengembalikan `0`). Hari dan tahun tidak begitu: `getDate()` mengembalikan 1 sampai 31 apa adanya. Untuk ditampilkan, bulan selalu perlu `+ 1`.

Method	Untuk `new Date(2026, 0, 31)`	Keterangan
`getFullYear()`	`2026`	tahun 4 digit
`getMonth()`	`0`	bulan, **mulai 0**
`getDate()`	`31`	tanggal 1–31
`getDay()`	`6`	hari dalam minggu: 0 Minggu sampai 6 Sabtu

Tanggal yang melewati batas tidak error, tetapi “meluber” ke bulan berikutnya. `new Date(2026, 1, 30)` (30 Februari) menjadi 2 Maret. Jika tanggal berasal dari input pengguna, validasi dulu.

Untuk memformat, gabungkan getter dengan `padStart` dari Modul 4:

```
function formatIso(tgl) {
  const bulan = String(tgl.getMonth() + 1).padStart(2, "0");   // +1 karena mulai dari 0
  const hari = String(tgl.getDate()).padStart(2, "0");
  return `${tgl.getFullYear()}/${bulan}/${hari}`;
}
console.log(formatIso(new Date(2026, 0, 5)));   // "2026/01/05"
```

Listing 10: format YYYY/MM/DD

2.7. throw, try, catch: Menangani Error

Untuk apa: menghentikan pekerjaan ketika ada kondisi yang tidak bisa dilanjutkan, lalu membiarkan pemanggil memutuskan apa yang dilakukan.

2.7.1. Melempar error

`throw new Error("pesan")` menghentikan fungsi **saat itu juga**. Baris setelahnya tidak dijalankan, dan fungsi tidak mengembalikan nilai. Error "naik" ke pemanggilnya, lalu ke pemanggil berikutnya, sampai ada yang menangkap. Jika tidak ada yang menangkap, program berhenti dan mencetak pesan error, persis seperti `TypeError` yang sudah sering kamu lihat.

```
function bagi(a, b) {
  if (b === 0) {
    throw new Error("tidak bisa dibagi 0");
  }
  return a / b;
}
```

Listing 11: throw

2.7.2. Menangkap error

```
try {
  console.log(bagi(6, 3));     // 2
  console.log(bagi(1, 0));     // melempar: sisa blok try dilewati
  console.log("tidak tercetak");
} catch (err) {
  console.log("tertangkap:", err.message);   // "tertangkap: tidak bisa dibagi 0"
} finally {
  console.log("selesai");      // selalu dijalankan, ada error atau tidak
}
```

Listing 12: try, catch, finally

Jalannya:

Urutan	Baris	Terjadi
1	`bagi(6, 3)`	mengembalikan 2, dicetak
2	`bagi(1, 0)`	`throw`, lompat ke `catch`
3	`console.log("tidak tercetak")`	**dilewati**
4	`catch (err)`	`err` berisi object Error, `err.message` adalah pesannya
5	`finally`	dijalankan

Object error punya `name` (misalnya `"Error"`, `"TypeError"`, `"SyntaxError"`) dan `message` (teks pesannya).

2.7.3. Di mana melempar, di mana menangkap

Pola yang umum:

- **Fungsi kecil yang dipakai di banyak tempat melempar error.** Fungsi itu tahu **apa** yang salah (qty tidak valid, data tidak ada), tetapi tidak tahu **bagaimana** kesalahan harus ditampilkan. Bisa jadi dicetak ke terminal, ditampilkan di halaman web, atau dikirim sebagai respons API.
- **Pemanggil di lapisan paling luar menangkap.** Pemanggil tahu konteksnya, jadi dia yang memutuskan bentuk pesannya.

Dengan pola ini, satu `try...catch` di luar cukup untuk menangkap error dari fungsi mana pun di dalamnya, sedalam apa pun pemanggilannya.

```
function validasiUmur(umur) {
  if (!Number.isInteger(umur) || umur < 0) {
    throw new Error("umur tidak valid");
  }
  return umur;
}

function daftar(nama, umur) {
  try {
    validasiUmur(umur);
    return `Terdaftar: ${nama}`;
  } catch (e) {
    return `GAGAL: ${e.message}`;
  }
}
console.log(daftar("Ani", 20));    // "Terdaftar: Ani"
console.log(daftar("Budi", -1));   // "GAGAL: umur tidak valid"
console.log(daftar("Cici", "20")); // "GAGAL: umur tidak valid"
```

Listing 13: validasi melempar, pemanggil menangkap

`Number.isInteger(x)` bernilai `true` hanya jika `x` bertipe number dan tidak punya desimal. `Number.isInteger("20")` bernilai `false`, karena `"20"` adalah string.

> **Catatan**
>
> Di Modul 5, kegagalan dikembalikan sebagai string `"GAGAL: ..."`. Itu cocok untuk method yang hasilnya memang pesan untuk pengguna. `throw` lebih cocok ketika fungsi seharusnya mengembalikan **data** (number, object, string struk), sehingga tidak ada nilai kembalian yang masuk akal saat gagal. Di TypeScript nanti (TS-5) ada pola ketiga, yaitu *Result*.

2.8. Membaca Pesan Error

Pesan error	Artinya	Penyebab umum
`does not provide an export named 'x'`	Nama yang di-import tidak di-export file itu.	Salah tulis nama, atau lupa `export`.
`ERR_MODULE_NOT_FOUND`	File yang di-import tidak ditemukan.	Lupa `.js`, atau path salah.
`require is not defined in ES module scope`	Memakai `require` di file module.	Contoh kode lama. Ganti dengan `import`.
`Cannot read properties of undefined (reading 'x')` setelah `map.get(...)`	Key tidak ada di Map.	Id salah, atau index belum diisi.
Pesan dari `throw new Error(...)` milikmu sendiri	Error yang kamu lempar tidak ditangkap.	Lupa `try...catch` di pemanggil.

> **Coba Sendiri**
>
> Kerjakan di fungsi `cobaSendiri()` pada file soal `Jawaban/soal-js6.js` (sudah tersedia di starter). Sebelum menjalankan, **tulis tebakanmu** sebagai komentar di ujung setiap baris `console.log`. Jalankan bagian ini saja dengan `node Jawaban/soal-js6.js coba`, lalu bandingkan.

```
console.log(Math.round(2.5), Math.round(-2.5), Math.floor(-2.5));
console.log(Math.max(...[3, 9, 4]), Math.max([3, 9, 4]));
const s = new Set(["a", "b", "a"]);
console.log(s.size, s.has("a"), [...s]);
const m = new Map([["x", 1]]);
console.log(m.get("x"), m.get("y"), m.size);
const d = new Date(2026, 0, 31);
console.log(d.getMonth(), d.getDate());
try {
  JSON.parse("{");
  console.log("lolos");
} catch (e) {
  console.log(e.name);
} finally {
  console.log("selesai");
}
```

1. Tebakan mana yang salah? Jelaskan penyebabnya.
2. Mengapa `Math.max([3, 9, 4])` menghasilkan `NaN`, sedangkan versi dengan `...` menghasilkan `9`?
3. Mengapa `"lolos"` tidak tercetak, tetapi `"selesai"` tercetak?
4. Tambahkan satu baris yang membuang duplikat dari `[3, 1, 3, 2, 1]` lalu mengurutkannya dari kecil, sehingga tercetak `[ 1, 2, 3 ]`.

Jawab pertanyaan di atas di bawah baris `// JAWABAN Coba Sendiri N` pada file soal.